LAPORAN PRAKTIKUM

PEMROGRAMAN BERORIENTASI OBJEK

MODUL 4 DAN 5
THE CORE OF OOP DAN ENCAPSULATION

Disusun Oleh:

Zakiati Latifa 105224033

PROGRAM STUDI ILMU KOMPUTER

FAKULTAS SAINS DAN KOMPUTER

UNIVERSITAS PERTAMINA

2026

I. Pendahuluan

Efisiensi operasional dalam manajemen perhotelan modern sangat bergantung pada keandalan sistem reservasi dalam menangani data dan validasi transaksi secara real-time. Permasalahan teknis seperti tamu yang melebihi batas muatan kamar hingga penyalahgunaan kode voucher sering kali menjadi celah yang merugikan pihak pengelola jika arsitektur sistem tidak dirancang dengan pengamanan data yang ketat. Oleh karena itu, diperlukan pengembangan perangkat lunak yang mampu mengotomatisasi pengecekan aturan bisnis guna meminimalisir kesalahan manusia (human error) dalam proses pelayanan tamu.

Dalam pemrograman berorientasi objek, prinsip Encapsulation (enkapsulasi) diterapkan untuk melindungi integritas data dengan cara menyembunyikan atribut krusial kelas dari akses langsung pihak luar. Melalui penggunaan modifier akses yang tepat, atribut seperti tipe kamar dan harga per malam hanya dapat dimodifikasi melalui metode Setter yang dilengkapi dengan logika validasi bisnis yang ketat. Pendekatan ini memastikan bahwa setiap data yang masuk, seperti tipe kamar "Reguler", "Premium", atau "Suite", telah sesuai dengan standar yang ditetapkan oleh sistem manajemen perhotelan.

Selain keamanan data, fleksibilitas sistem ditingkatkan melalui implementasi konsep Method Overloading pada fungsi pemesanan dan penagihan biaya. Dengan overloading, satu metode dapat memiliki perilaku yang berbeda-beda, misalnya melakukan pemesanan kamar sederhana atau melakukan validasi jumlah tamu terhadap kapasitas maksimal kamar secara bersamaan. Integrasi metode ini juga memungkinkan departemen keuangan untuk menghitung total tagihan secara otomatis, baik menggunakan tarif standar maupun melalui perhitungan diskon voucher "PROMO" dengan syarat durasi menginap tertentu.

II. Variabel

No	Nama Variabel	Tipe Data	Fungsi

III. Constructor dan Method

No	Nama Metode	Jenis Metode	Fungsi
1	KamarHotel	Constructor 1	Digunakan untuk mendaftarkan kamar baru dengan harga yang

			belum ditentukan. Konstruktor ini menggunakan 3 parameter
2	KamarHotel	Constructor 2	Digunakan untuk kamar yang sudah terdaftar dan harga sudah diketahui sejak awal
3	getNomorKamar	Metode	Digunakan untuk membaca nomor kamar tanpa bisa mengubahnya (tidak ada setter)
4	getTipeKamar	Metode	Digunakan untuk membaca tipe kamar yang udh divalidasi oleh setter
5	setTipeKamar	Metode	Digunakan untuk mengubah tipe kamar dengan validasi
6	getKapasitasMaksimal	Metode	Digunakan untuk membaca kapasitas kamar maksimal tamu yang diperbolehkan
7	getHargaPerKamar	Metode	Digunakan untuk membaca harga kamar per malam
8	setHargaPerKamar	Metode	Digunakan untuk mengubah harga kamar dengan validasi
9	isTersedia	Metode	Digunakan untuk membaca status ketersediaan kamar
10	pesanKamar	Metode 1	Digunakan untuk memesan kamar tanpa parameter
11	pesanKamar	Metode 2	Digunakan untuk memesan kamar dengan validasi jumlah tamu
12	batalPesan	Metode	Digunakan untuk membatalkan pemesanan kamar
13	hitungTotalBayar	Metode 1	Digunakan untuk menghitung total tagihan jumlah malam parameter jumlah malam

14	hitungTotalBayar	Metode 2	Digunakan untuk menghitung tagihan jumlah malam yang divalidasi dan kode voucher
15	profilKamar	Metode	Digunakan untuk mencetak seluruh informasi kamar dan total tagihan dari pesanan

IV. Dokumentasi dan Pembahasan Code

1. Pertama membuat class dengan nama KamarHotel dengan menerapkan encapsulation.

```
public class KamarHotel {
```

2. Lalu kita akan membuat seluruh atribut dengan modifier private sebagai penarapan prinsip encapsulation dalam OOP yang bertujuan agar atribut tidak dapat diakses atau diubah langsung dari luar class.

```
    private String nomorKamar;
    private String tipeKamar;
    private int kapasitasMaksimal;
    private double hargaPerKamar;
    private boolean isTersedia;
```

nomorKamar bertipe String yang menyimpan identitas unik kamar hotel. tipeKamar bertipe String yang menyimpan kategori kamar. kapasitasMaksimal bertipe int yang menyimpan batas maksimal jumlah tamu yang diperbolehkan menginap. hargaPerKamar bertipe double yang menyimpan harga sewa kamar per malam. isTersedia bertipe boolean yang menyimpan status ketersediaan kamar.

3. Atribut totalTagihan bertipe double yang menyimpan hasil perhitungan total tagihan kamar setelah method hitungTotalBayar2() dipanggil

```
    double totalTagihan;
```

4. Selanjutnya membuat konstruktor pertama menerima 3 parameter yaitu nomorKamar, tipeKamar, dan kapasitasMaksimal. Konstruktor ini digunakan ketika kamar baru didaftarkan ke sistem namun harga belum ditentukan.

```java
public KamarHotel(String nomorKamar, String tipeKamar, int kapasitasMaksimal){
    this.nomorKamar = nomorKamar;
    this.tipeKamar = tipeKamar;
    this.kapasitasMaksimal = kapasitasMaksimal;
    this.hargaPerKamar = 0;
    this.isTersedia = true;
}
```

5. Selanjutnya membuat konstruktor kedua menerima 4 parameter yaitu nomorKamar, tipeKamar, kapasitasMaksimal, dan hargaPerKamar. Konstruktor ini digunakan ketika kamar didaftarkan ke sistem dan harga sudah diketahui sejak awal, sehingga semua data langsung lengkap dalam satu langkah.

```java
public KamarHotel(String nomorKamar, String tipeKamar, int kapasitasMaksimal, double hargaPerKamar){
    this.nomorKamar = nomorKamar;
    this.tipeKamar = tipeKamar;
    this.kapasitasMaksimal = kapasitasMaksimal;
    this.hargaPerKamar = hargaPerKamar;
    this.isTersedia = true;
}
```

6. getNomorKamar() adalah method getter yang mengembalikan nilai atribut nomorKamar bertipe String. Method ini tidak memiliki setter karena nomor kamar bersifat tetap dan tidak boleh diubah setelah objek dibuat. cara mengisi nomorKamar adalah melalui konstruktor.

```java
public String getNomorKamar(){
    return nomorKamar;
}
```

7. getTipeKamar() adalah method getter yang mengembalikan nilai atribut tipeKamar bertipe String.

```java
public String getTipeKamar(){
    return tipeKamar;
}
```

8. setTipeKamar() adalah method setter yang bertugas mengubah nilai atribut tipeKamar dengan validasi ketat. Method menerima satu parameter tipeKamar bertipe String dan tidak mengembalikan nilai (void). Method menggunakan equalsIgnoreCase() untuk membandingkan input tanpa memperhatikan huruf besar atau kecil, sehingga input seperti "regular", "REGULAR", atau "Regular" tetap diterima. Jika input valid ("Regular", "Premium", atau "Suite"), nilai tipeKamar langsung diubah sesuai input yang diberikan. Jika input tidak valid, sistem mencetak pesan peringatan ke layar dan secara otomatis memaksa nilai tipeKamar menjadi "Regular" sebagai nilai default.

```java
    public void setTipeKamar(String tipeKamar){
        if(tipeKamar.equalsIgnoreCase(anotherString: "Regular") || tipeKamar.equalsIgnoreCase(anotherString: "Premium")
            || tipeKamar.equalsIgnoreCase(anotherString: "Suite")){
            this.tipeKamar = tipeKamar;
        } else {
            System.out.println(x: "Tipe kamar tidak tersedia || Tipe kamar Regular untuk Anda");
            this.tipeKamar = "Regular";
        }
    }
```

9. getKapasitasMaksimal() mengembalikan nilai kapasitasMaksimal bertipe int. Tidak memiliki setter karena kapasitas kamar bersifat tetap dan hanya bisa diisi melalui konstruktor.

```java
    public int getKapasitasMaksimal(){
        return kapasitasMaksimal;
    }
```

10. getHargaPerKamar() mengembalikan nilai hargaPerKamar bertipe double. Memiliki setter setHargaPerKamar() dengan validasi minimum 50000, sehingga nilai yang dikembalikan selalu valid.

```java
    public double getHargaPerKamar(){
        return hargaPerKamar;
    }
```

11. setHargaPerKamar() adalah method setter yang mengubah nilai hargaPerKamar dengan validasi minimum. Method menerima satu parameter bertipe double dan tidak mengembalikan nilai (void).

```java
    public void setHargaPerKamar(double hargaPerKamar){
        if(hargaPerKamar < 50000){
            System.out.println(x: "Harga per kamar tidak valid.");
            this.hargaPerKamar = 50000;
        } else {
            this.hargaPerKamar = hargaPerKamar;
        }
    }
```

12. isTersedia() adalah method getter yang mengembalikan nilai atribut isTersedia bertipe boolean. Menggunakan prefix **is** bukan get sesuai konvensi standar Java untuk atribut bertipe boolean. Nilai yang dikembalikan adalah true jika kamar tersedia dan false jika kamar sudah terisi. Nilai ini berubah secara otomatis melalui method pesenKamar() dan batalPesan(), bukan melalui setter langsung.

```java
    public boolean isTersedia(){
        return isTersedia;
    }
```

13. Method pemesanan kamar tanpa parameter. Jika kamar tersedia, status isTersedia langsung diubah menjadi false dan mengembalikan pesan berhasil. Jika sudah terisi, mengembalikan pesan penolakan.

```java
    public String pesanKamar(){
        if(isTersedia){
            isTersedia = false;
            return "Kamar berhasil terisi";
        } else {
            return "Maaf, kamar tidak tersedia!";
        }
    }
```

14. Method pemesanan kamar dengan validasi jumlah tamu. Jika jumlah tamu melebihi kapasitasMaksimal, pemesanan ditolak dan status kamar tetap true. Kedua method pesenKamar() merupakan penerapan Method Overloading.

```java
    public String pesanKamar(int jumlahTamu){
        if(isTersedia){
            if(jumlahTamu > kapasitasMaksimal){
                return "Maaf jumlah tamu Overcapacity!";
            } else {
                isTersedia = false;
                return "Kamar berhasil terisi";
            }
        } else {
            return "Maaf, kamar tidak tersedia!";
        }
    }
```

15. Method pembatalan pemesanan. Jika kamar sedang terisi (isTersedia = false), status dikembalikan menjadi true. Jika kamar sudah tersedia, mencetak pesan bahwa kamar memang sudah tersedia.

```java
    public void batalPesan(){
        if(!isTersedia){
            isTersedia = true;
            System.out.println(x: "Pesanan kamar berhasil di batalkan");
        } else {
            System.out.println(x: "Kamar sudah tersedia");
        }
    }
```

16. Method perhitungan tagihan standar tanpa diskon. Mengembalikan hasil perkalian jumlahMalam dengan hargaPerKamar. Jika jumlahMalam kurang dari 1, mengembalikan nilai 0.

```java
    public double hitungTotalBayar(int jumlahMalam){
        if(jumlahMalam < 1){
            return 0;
        } else {
            return jumlahMalam * hargaPerKamar;
        }
    }
```

17. Method perhitungan tagihan dengan voucher diskon. Jika kode voucher "PROMO" dan menginap minimal 3 malam, diberikan diskon 20% dari total tagihan. Jika syarat tidak terpenuhi, mengembalikan tagihan normal tanpa diskon. Kedua method hitungTotalBayar() merupakan penerapan Method Overloading.

```java
    public String hitungTotalBayar(int jumlahMalam, String kodeVoucher){
        totalTagihan = jumlahMalam * hargaPerKamar;
        if(kodeVoucher.equalsIgnoreCase(anotherString: "PROMO") && jumlahMalam >= 3){
            double diskon = totalTagihan * 0.2;
            totalTagihan = totalTagihan - diskon;
            return "Total Tagihan: " + totalTagihan;
        } else {
            return "Total Pembayaran: " + totalTagihan;
        }
    }
```

18. Method untuk mencetak seluruh informasi kamar secara lengkap dan rapi ke layar dalam bentuk struk. Menampilkan semua atribut kamar termasuk status ketersediaan dan total tagihan terakhir yang dihitung.


```java
    public String hitungTotalBayar(int jumlahMalam, String kodeVoucher){
        totalTagihan = jumlahMalam * hargaPerKamar;
        if(kodeVoucher.equalsIgnoreCase(anotherString: "PROMO") && jumlahMalam >= 3){
            double diskon = totalTagihan * 0.2;
            totalTagihan = totalTagihan - diskon;
            return "Total Tagihan: " + totalTagihan;
        } else {
            return "Total Pembayaran: " + totalTagihan;
        }
    }
```


19. Selanjutnya buat class ke dua yaitu main untuk melakukan uji dari class KamarHotel

```java
public class App {
    Run | Debug
    public static void main(String[] args){
        KamarHotel kamar1 = new KamarHotel(nomorKamar: "101", tipeKamar: "Regular", kapasitasMaksimal: 2);
        kamar1.setTipeKamar(tipeKamar: "Presidential");
        kamar1.setHargaPerKamar(-10000);

        KamarHotel kamar2 = new KamarHotel(nomorKamar: "202", tipeKamar: "Suite", kapasitasMaksimal: 2, harg… 100000);
        kamar2.pesanKamar(jumlahTamu: 4);
        kamar2.pesanKamar(jumlahTamu: 2);
        kamar2.pesanKamar();

        kamar1.hitungTotalBayar(jumlahMalam: 2, kodeVoucher: "PROMO");
        kamar2.hitungTotalBayar(jumlahMalam: 4, kodeVoucher: "PROMO");

        kamar1.profilKamar();
        kamar2.profilKamar();

    }
}
```

IV. Kesimpulan

Berdasarkan hasil implementasi sistem reservasi hotel, dapat disimpulkan bahwa penerapan prinsip *Encapsulation* melalui penggunaan *modifier* privat dan validasi ketat pada metode *Setter* berhasil menjaga integritas data. Sistem secara otomatis mampu menangani kesalahan operasional dengan memaksa input tipe kamar yang tidak valid menjadi "Reguler" serta menetapkan batas minimum harga sebesar Rp50.000 untuk mencegah kerugian finansial.

Selain itu, penggunaan *Method Overloading* pada fungsi pemesanan dan penagihan memberikan fleksibilitas tinggi dalam menangani berbagai skenario transaksi. Sistem terbukti tangguh terhadap *human error* karena mampu memblokir pesanan yang melebihi kapasitas maksimal kamar serta melakukan validasi otomatis terhadap syarat durasi menginap untuk penggunaan kode voucher promo.

IV. Daftar Pustaka

Deitel, P., & Deitel, H. (2017). *Java How to Program, Early Objects (11th ed.)*. Pearson.

Schildt, H. (2018). *Java: The Complete Reference (11th ed.)*. McGraw-Hill Education.

Wu, C. T. (2010). *An Introduction to Object-Oriented Programming with Java (5th ed.)*.